

This is a manual for converting a .pt trained model to an .trt engine type on Google Colab.

PT → ONNX.

Step-1: Go to [Google-Colab](https://colab.research.google.com/) and open a new notebook

Step-2: Upload my_model.pt to colab. (remane your model to my_model.pt or change to your model name in the code below)

Add the following codeblocks one by one

Block-1:

```
!pip install --upgrade pip
```

Block-2:

```
# Install latest directly from GitHub
```

```
!pip install git+https://github.com/ultralytics/ultralytics.git@main
```

Block-3:

```
import torch
```

```
import onnx
```

```
import onnx_graphsurgeon as gs
```

```
import numpy as np
```

```
from ultralytics import YOLO
```

```
# --- LOAD MODEL ---
```

```
model = YOLO("/content/my_model.pt")
```

```
model = model.model.eval()
```

```
# Dummy input
```

```
dummy = torch.randn(1, 3, 416, 416, device="cpu")
```

```
# --- EXPORT TO ONNX ---
```

```
onnx_path = "/content/my_model.onnx"
```

```
torch.onnx.export(
```

```
    model,
```

```
    dummy,
```

```
    onnx_path,
```

```
    input_names=["images"],
```

```

    output_names=["output0"],
    opset_version=12,
    do_constant_folding=True,
    dynamic_axes=None # static for TensorRT
)
print(f'ONNX model saved to {onnx_path}')

# --- FIX INT64 TO INT32 ---
onnx_model = onnx.load(onnx_path)
graph = gs.import_onnx(onnx_model)

changed = 0
for node in graph.nodes:
    for inp in node.inputs:
        if inp.dtype == np.int64:
            inp.dtype = np.int32
            changed += 1
    for out in node.outputs:
        if out.dtype == np.int64:
            out.dtype = np.int32
            changed += 1

fixed_path = "/content/my_model_fixed.onnx"
onnx.save(gs.export_onnx(graph), fixed_path)
print(f'Fixed {changed} INT64 tensors → INT32')
print(f'Clean ONNX saved to {fixed_path}')

# --- TEST LOAD ---
onnx.checker.check_model(onnx.load(fixed_path))
print("✅ ONNX model passed validation")

# --- DOWNLOAD TO LOCAL MACHINE ---

```

```
from google.colab import files
```

```
files.download(fixed_path)
```

Block-4: (Optional)

```
import onnx
```

```
m = onnx.load("/content/my_model_fixed.onnx")
```

```
int64_nodes = []
```

```
for n in m.graph.node:
```

```
    for a in list(n.input) + list(n.output):
```

```
        for t in m.graph.value_info:
```

```
            if t.name == a and t.type.tensor_type.elem_type == onnx.TensorProto.INT64:
```

```
                int64_nodes.append((n.name, a))
```

```
print("Found INT64 tensors:", int64_nodes)
```

Run these block in order. my_model.onnx will be downloaded automatically and ready to be exported to Jetson Nano.

ONNX → TRT

Step-1: Export the onnx file to jetson nano

Step-2: Install PyCuda by following cmd lines

```
pip install 'https://github.com/jetson-nano-wheels/python3.6-pycuda-2021.1/releases/download/v0.0.1/pycuda-2021.1-cp36-cp36m-linux_aarch64.whl'
```

(If you have followed week_12 report, you already have all the dependencies for PyCuda)

Step-3: Create a .py file by using nano text editor and paste this code inside

```
import tensorrt as trt
```

```
import pycuda.driver as cuda
```

```
import pycuda.autoinit
```

```
import os
```

```
onnx_file_path = "/path/to/your/.onnx"
```

```
trt_file_path = "/path/to/where/you want to save/my_model.trt"
```

```
TRT_LOGGER = trt.Logger(trt.Logger.WARNING)
```

```
def build_engine(onnx_path, trt_path, max_batch_size=1, fp16_mode=True):
```

```
    with trt.Builder(TRT_LOGGER) as builder, \
```

```
        builder.create_network(1 <<
```

```
int(trt.NetworkDefinitionCreationFlag.EXPLICIT_BATCH)) as network, \
```

```
        trt.OnnxParser(network, TRT_LOGGER) as parser:
```

```
    # Create builder config
```

```
    config = builder.create_builder_config()
```

```
    config.max_workspace_size = 1 << 30 # 1GB
```

```
    if fp16_mode:
```

```
        config.set_flag(trt.BuilderFlag.FP16)
```

```
    # Parse ONNX
```

```
with open(onnx_path, "rb") as f:
    if not parser.parse(f.read()):
        print("ERROR: Failed to parse ONNX model")
        for i in range(parser.num_errors):
            print(parser.get_error(i))
        return None
```

```
# Build engine
engine = builder.build_engine(network, config)
if engine is None:
    print("ERROR: Failed to build engine")
    return None
```

```
# Serialize engine
with open(trt_path, "wb") as f:
    f.write(engine.serialize())
print(f"TRT engine saved to: {trt_path}")
return engine
```

```
if __name__ == "__main__":
    build_engine(onnx_file_path, trt_file_path)
```

Save it and run it using python3